

CS3.306: Algorithms and Operating Systems

Lecture 3 & 4: Process Virtualization - API

INSTRUCTOR: DR. DEEPAK GANGADHARAN

What features should the OS provide?

- Create a process
- Destroy a process
- Wait
- Suspend a process
- Retrieve status of a process

How to achieve the features? – System calls!

- Way for the user program to interact with the OS
- OS provides some functions available as “System Calls”
- Function call into OS code that runs at a higher privilege level

Some System Calls

File Management	Process Management	Communication	Protection
<code>fd =open(file,..)</code>	<code>fork()</code>	<code>Pipe()</code>	<code>chmod()</code>
<code>close(fd)</code>	<code>wait()</code>	<code>Shmget()</code>	<code>Unmask()</code>
<code>write(fd, ...)</code>	<code>exec()</code>	<code>Mmap()</code>	<code>chown()</code>
...	...	...	...

System Calls for Process (Unix)

System Call	Role
fork()	Creates a new child process
exec()	Executes a process
wait()	Blocks a parent to until the child terminates
exit()	Terminates a process

- Many variants of the system calls exist

Fork System Call

- Used to create a new process
- rc has the child process id
- getpid() gets the parent process id
- Results in two processes running at the same time
 - child is copy of the parent process

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main(int argc, char *argv[]) {
6      printf("hello (pid:%d)\n", (int) getpid());
7      int rc = fork();
8      if (rc < 0) {
9          // fork failed
10         fprintf(stderr, "fork failed\n");
11         exit(1);
12     } else if (rc == 0) {
13         // child (new process)
14         printf("child (pid:%d)\n", (int) getpid());
15     } else {
16         // parent goes down this path (main)
17         printf("parent of %d (pid:%d)\n",
18             rc, (int) getpid());
19     }
20     return 0;
21 }
```

Fork System Call

- Child is an almost exact copy of the parent
 - Gets a copy of the open file descriptors, network connections and other resources
- Both *parent* and *child* start execution right after `fork()`
- Both *parent* and *child* modify memory independently
- The order of execution is not **deterministic**

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main(int argc, char *argv[]) {
6      printf("hello (pid:%d)\n", (int) getpid());
7      int rc = fork();
8      if (rc < 0) {
9          // fork failed
10         fprintf(stderr, "fork failed\n");
11         exit(1);
12     } else if (rc == 0) {
13         // child (new process)
14         printf("child (pid:%d)\n", (int) getpid());
15     } else {
16         // parent goes down this path (main)
17         printf("parent of %d (pid:%d)\n",
18               rc, (int) getpid());
19     }
20     return 0;
21 }
```

Fork System Call

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main(int argc, char *argv[]) {
6      printf("hello (pid:%d)\n", (int) getpid());
7      int rc = fork();
8      if (rc < 0) {
9          // fork failed
10         fprintf(stderr, "fork failed\n");
11         exit(1);
12     } else if (rc == 0) {
13         // child (new process)
14         printf("child (pid:%d)\n", (int) getpid());
15     } else {
16         // parent goes down this path (main)
17         printf("parent of %d (pid:%d)\n",
18             rc, (int) getpid());
19     }
20     return 0;
21 }
```

When you run this program, you get the following output

```
hello (pid:29146)
parent of 29147 (pid:29146)
child (pid:29147)
```


Wait System Call

- `wait()` call blocks in parent until the child process terminates (options like **`waitpid()`** exists)
- `wait()` makes the output **deterministic**.
- **Behaviour of wait**
 - If the parent process has multiple children, the `wait()` call will cause the parent to wait until any child process terminates.
 - If a child process terminates, `wait()` returns the PID of the terminated child.
 - If multiple child processes terminate, `wait()` will use any one of them arbitrarily and return its PID.
 - If no children exist, `wait()` immediately returns -1.
- **Wait allows OS to reclaim the resources of the child – Prevent zombies**

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main(int argc, char *argv[]) {
7      printf("hello (pid:%d)\n", (int) getpid());
8      int rc = fork();
9      if (rc < 0) {          // fork failed; exit
10         fprintf(stderr, "fork failed\n");
11         exit(1);
12     } else if (rc == 0) { // child (new process)
13         printf("child (pid:%d)\n", (int) getpid());
14     } else {               // parent goes down this path
15         int rc_wait = wait(NULL);
16         printf("parent of %d (rc_wait:%d) (pid:%d)\n",
17               rc, rc_wait, (int) getpid());
18     }
19     return 0;
20 }
21
```

Wait System Call

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main(int argc, char *argv[]) {
7      printf("hello (pid:%d)\n", (int) getpid());
8      int rc = fork();
9      if (rc < 0) {                // fork failed; exit
10         fprintf(stderr, "fork failed\n");
11         exit(1);
12     } else if (rc == 0) { // child (new process)
13         printf("child (pid:%d)\n", (int) getpid());
14     } else {                    // parent goes down this path
15         int rc_wait = wait(NULL);
16         printf("parent of %d (rc_wait:%d) (pid:%d)\n",
17               rc, rc_wait, (int) getpid());
18     }
19     return 0;
20 }
21
```

When you run this program, you get the following output

```
hello (pid:29266)
child (pid:29267)
parent of 29267 (rc_wait:29267) (pid:29266)
```

Exec System Call

- `exec()` allows the child to run a different program unlike the parent.
- Loads the code of the executable and overwrites its current code segment.
- OS then runs that program passing in any arguments as the `argv` of that process.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6
7  int main(int argc, char *argv[]) {
8      printf("hello (pid:%d)\n", (int) getpid());
9      int rc = fork();
10     if (rc < 0) {          // fork failed; exit
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     } else if (rc == 0) { // child (new process)
14         printf("child (pid:%d)\n", (int) getpid());
15         char *myargs[3];
16         myargs[0] = strdup("wc"); // program: "wc"
17         myargs[1] = strdup("p3.c"); // arg: input file
18         myargs[2] = NULL;          // mark end of array
19         execvp(myargs[0], myargs); // runs word count
20         printf("this shouldn't print out");
21     } else {                // parent goes down this path
22         int rc_wait = wait(NULL);
23         printf("parent of %d (rc_wait:%d) (pid:%d)\n",
24               rc, rc_wait, (int) getpid());
25     }
26     return 0;
27 }
```

Exec System Call

- Child process calls `execvp()` to run the program `WC`
- Gives the number of lines, words and bytes in the program

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6
7  int main(int argc, char *argv[]) {
8      printf("hello (pid:%d)\n", (int) getpid());
9      int rc = fork();
10     if (rc < 0) {          // fork failed; exit
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     } else if (rc == 0) { // child (new process)
14         printf("child (pid:%d)\n", (int) getpid());
15         char *myargs[3];
16         myargs[0] = strdup("wc"); // program: "wc"
17         myargs[1] = strdup("p3.c"); // arg: input file
18         myargs[2] = NULL;          // mark end of array
19         execvp(myargs[0], myargs); // runs word count
20         printf("this shouldn't print out");
21     } else {                  // parent goes down this path
22         int rc_wait = wait(NULL);
23         printf("parent of %d (rc_wait:%d) (pid:%d)\n",
24               rc, rc_wait, (int) getpid());
25     }
26     return 0;
27 }
```

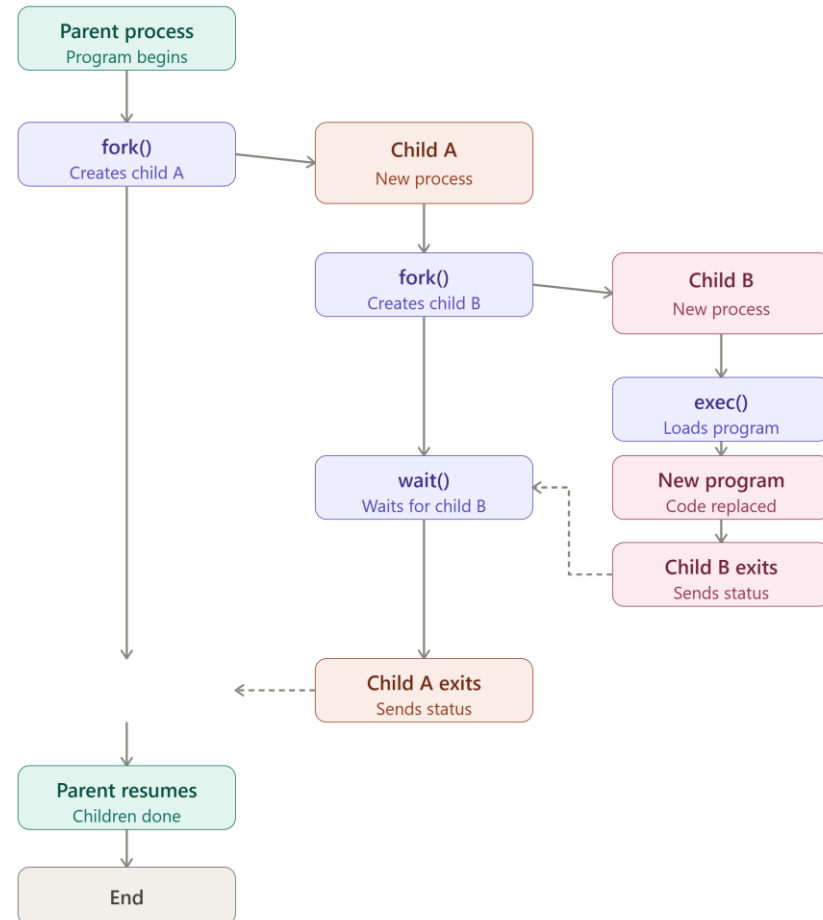
Exec System Call

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6
7  int main(int argc, char *argv[]) {
8      printf("hello (pid:%d)\n", (int) getpid());
9      int rc = fork();
10     if (rc < 0) {                // fork failed; exit
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     } else if (rc == 0) { // child (new process)
14         printf("child (pid:%d)\n", (int) getpid());
15         char *myargs[3];
16         myargs[0] = strdup("wc"); // program: "wc"
17         myargs[1] = strdup("p3.c"); // arg: input file
18         myargs[2] = NULL; // mark end of array
19         execvp(myargs[0], myargs); // runs word count
20         printf("this shouldn't print out");
21     } else { // parent goes down this path
22         int rc_wait = wait(NULL);
23         printf("parent of %d (rc_wait:%d) (pid:%d)\n",
24             rc, rc_wait, (int) getpid());
25     }
26     return 0;
27 }
```

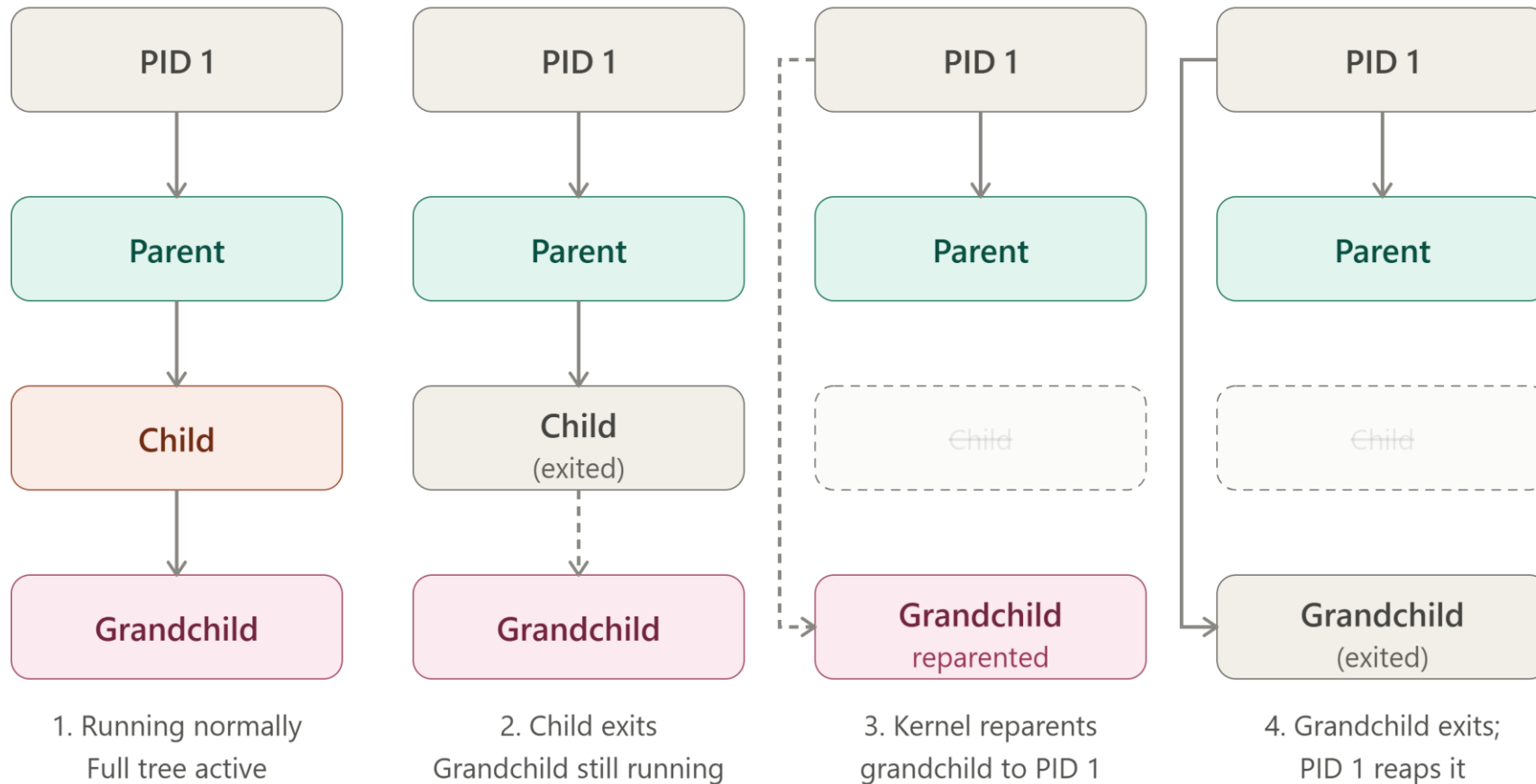
When you run this program, you get the following output

```
hello (pid:29383)
child (pid:29384)
      29      107      1030      p3.c
parent of 29384 (rc_wait:29384) (pid:29383)
```

Flow Diagram – Process Creation and Execution



Multi-Stage Process Hierarchy Diagram – What happens when a child exits?



Why all these APIs – How does a Shell work?

- Shell does the following
 - Read user command
 - Forks a child and exec the command
 - Waits for it to finish and proceeds to next command



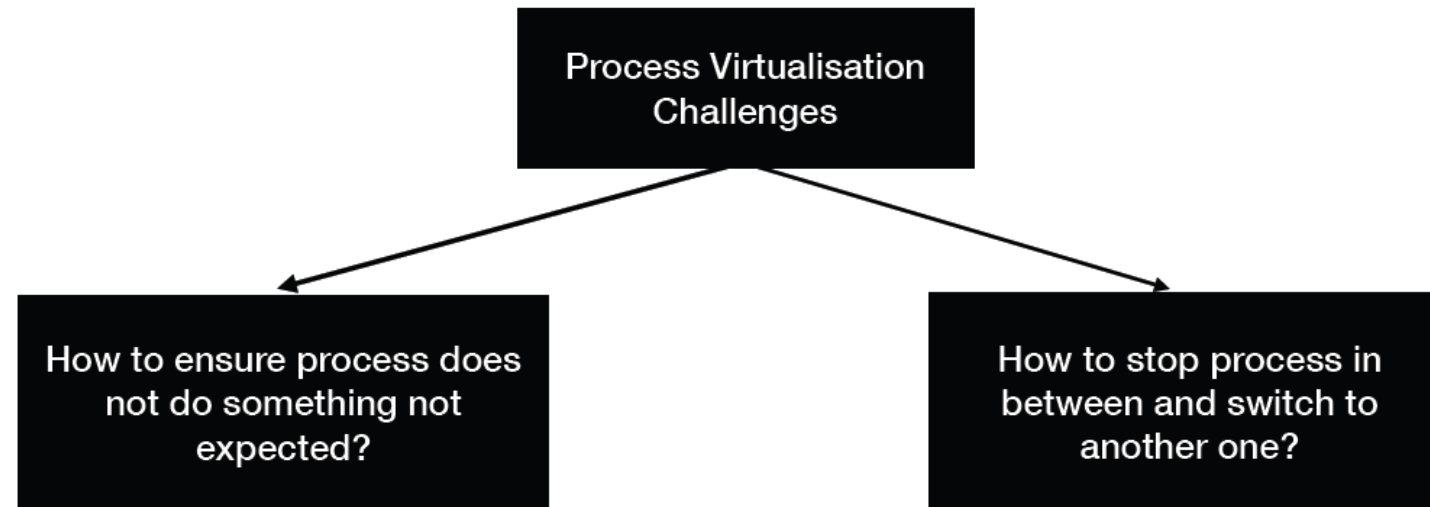
How does a shell work?

prompt > wc process_3.c > out.txt

- Shell will fork a child
 - Redirects its standard output to text file (out.txt)
 - Calls exec() on the child
 - The output will be redirected to out.txt

Question that needs to be answered - How to run multiple processes?

Two major problems to be solved



What if we allow process to do whatever it wants?

How can multiple processes run?

- Hardware support
 - Have some low level mechanisms to switch process
 - Challenges – Performance Overhead?
- Software support
 - Have some policies which decides what needs to be executed
 - Challenges – control overhead

Is this enough?

OS	Program
1. Create an entry in process list 2. Allocate memory for the program 3. Load program into memory 4. Setup stack with argc/argv 5. Clear registers 6. Execute call main()	
	7. Run main () 8. Execute return from main()
9. Free memory of process 10. Remove process from process list	

Thank You!!